Practical no 1

Comprehensive NLP Pipeline for Linguistic Analysis Using spaCy and NLTK

Introduction to NLP:

1. What is NLP?

Natural Language Processing (**NLP**) is a field of **Artificial Intelligence (AI)** and **Linguistics** that focuses on enabling computers to understand, interpret, and generate human language.
It's the technology behind things like:

- Chatbots (e.g., ChatGPT, Gemini)
- Voice assistants (Siri, Alexa)
- Machine translation (Google Translate)
- Sentiment analysis (detecting emotions in text)

2. Why is NLP important?

Human language is:

- **Complex** (grammar, meaning, context)
- **Ambiguous** (same word can have multiple meanings)
- **Variable** (different styles, dialects, slang)

NLP allows machines to:

- Extract **meaning** from text or speech
- Communicate with humans in a **natural** way
- Process **large volumes** of language data efficiently

3. Main Tasks in NLP

Some core tasks include:

- **Tokenization:** Splitting text into words or sentences
- **Part-of-Speech Tagging:** Identifying nouns, verbs, etc.
- **Named Entity Recognition (NER):** Detecting names, places, dates
- **Sentiment Analysis:** Identifying opinions/emotions
- **Machine Translation:** Converting text from one language to another
- **Text Summarization:** Creating concise summaries
- **Speech Recognition:** Converting spoken language to text

4. How NLP Works

NLP combines:

1. **Linguistics** – Grammar, syntax, semantics
2. **Machine Learning (ML)** – Learning patterns from data
3. **Deep Learning (DL)** – Using neural networks to capture context and meaning

Common libraries & frameworks:

- **NLTK**, **spaCy** – For basic NLP processing
- **Transformers** (Hugging Face) – For advanced deep learning models like **BERT** and **GPT**
- **SpeechRecognition** – For speech-to-text tasks

5. Applications of NLP

- **Search engines** (Google search suggestions)
- **Customer support** (chatbots, email classification)
- **Healthcare** (analyzing medical records)
- **Social media monitoring** (detecting trends and opinions)

NLP Pipeline Flowchart:

1. Sentence Segmentation

Sentence segmentation is the process of breaking a large text or paragraph into individual sentences. This is an essential step because most NLP tasks are performed on a sentence-by-sentence basis.
Example:
Input: "I love programming. It makes me happy."
Output: ["I love programming.", "It makes me happy."]

---

2. Word Tokenization

Word tokenization splits each sentence into smaller units called *tokens*, which may be words, punctuation marks, or subwords.
Example:
Sentence: "I love programming."
Tokens: ["I", "love", "programming", "."]

---

3. Stemming

Stemming reduces words to their base or root form by removing suffixes and prefixes, often without considering grammar rules. It may produce non-dictionary words.
Example:
"playing" → "play"
"studies" → "studi"

---

4. Lemmatization

Lemmatization also reduces words to their base form but uses vocabulary and grammar rules, ensuring the result is a valid dictionary word.
Example:
"playing" → "play"
"better" → "good"

---

5. Identifying Stop Words

Stop words are commonly used words (such as "is", "and", "the") that usually carry less important meaning in text analysis. This step involves detecting and optionally removing them to reduce noise.
Example:
Sentence: "I am learning NLP"
After removing stop words: ["learning", "NLP"]

---

6. Dependency Parsing

Dependency parsing analyzes the grammatical structure of a sentence by identifying the relationships between words.
Example:
Sentence: "I love programming."

- "love" → main verb
- "I" → subject of "love"
- "programming" → object of "love"

---

7. Part-of-Speech (POS) Tagging

This step assigns each token a grammatical role, such as noun, verb, adjective, etc. POS tagging is essential for syntactic analysis.
Example:
Sentence: "The cat sleeps."
Tags: [("The", DET), ("cat", NOUN), ("sleeps", VERB)]

---

8. Named Entity Recognition (NER)

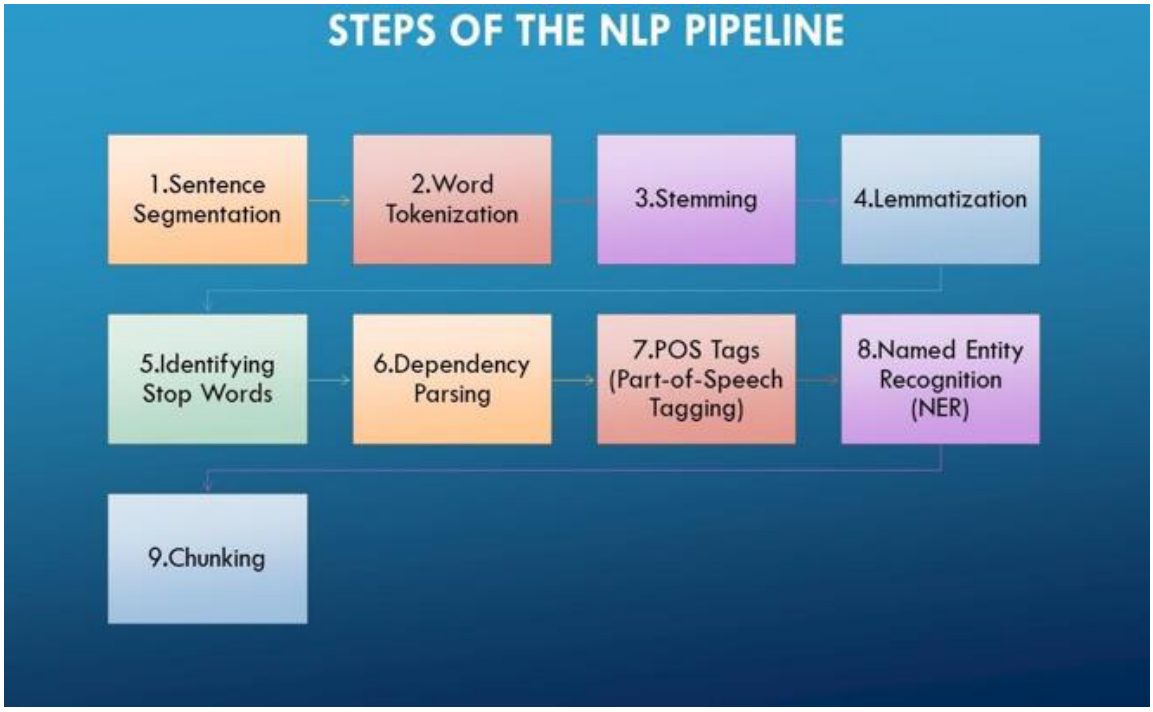
NER identifies and classifies named entities such as people, organizations, locations, dates, and more.
Example:
Sentence: "Apple Inc. is based in California."
NER Output:

- Apple Inc. → Organization
- California → Location

---

9. Chunking

Chunking groups tokens or words into meaningful phrases (such as noun phrases or verb phrases) based on POS tags and grammar patterns.
Example:
Sentence: "The big brown dog"
Noun Phrase (NP) Chunk: ["The big brown dog"]



Code:

1. Importing Libraries:

```
# Practical no 1 : Comprehensive NLP Pipeline for Linguistic Analysis
import spacy
import nltk
from nltk.stem import PorterStemmer
```

2. Loading Models and Resources:

```
nlp = spacy.load("en_core_web_sm")
nltk.download('punkt')
stemmer = PorterStemmer()
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

3. Defining the Input Text:

```
[4] text = ("Operation Sindoor was carried out on May 7, 2025, between 1:04 AM and 1:24
```

4. Processing Text with spaCy:

```
[5] # Process text
    doc = nlp(text)
```

5. Tokenization:

```
[8] # 1. Tokenization
    print("1 Tokens:")
    print([token.text for token in doc])
```

```
1 Tokens:
['Operation', 'Sindoor', 'was', 'carried', 'out', 'on', 'May', '7', ',', '2025',
```

6. Stopword Removal:

7. Stemming:

```
print("\n Stemming:")
for token in filtered_tokens:
    print(f"{token:50} ➤ {stemmer.stem(token)}")
```

```
 Stemming:
Operation                                          ➤ oper
Sindoor                                            ➤ sindoor
carried                                            ➤ carri
involved                                           ➤ involv
precision                                          ➤ precis
strikes                                            ➤ strike
terrorist                                          ➤ terrorist
camps                                              ➤ camp
located                                            ➤ locat
Pakistan                                           ➤ pakistan
Pakistan                                           ➤ pakistan
occupied                                           ➤ occupi
Jammu                                              ➤ jammu
Kashmir                                            ➤ kashmir
PoJK                                               ➤ pojk
strikes                                            ➤ strike
targeted                                           ➤ target
headquarters                                       ➤ headquart
groups                                             ➤ group
```

8. Lemmatization:

```
print("\n Lemmatization:")
for token in doc:
    print(f"{token.text:20} ➤ {token.lemma_}")
```

```
 Lemmatization:
Operation            ➤ Operation
Sindoor              ➤ Sindoor
was                  ➤ be
carried              ➤ carry
out                  ➤ out
on                   ➤ on
May                  ➤ May
7                    ➤ 7
,                    ➤ ,
2025                 ➤ 2025
,                    ➤ ,
between              ➤ between
1:04                 ➤ 1:04
AM                   ➤ am
and                  ➤ and
1:24                 ➤ 1:24
AM                   ➤ am
.                    ➤ .
It                   ➤ it
involved             ➤ involve
precision            ➤ precision
strikes              ➤ strike
against              ➤ against
```

9. POS Tagging:

```
print("\n POS Tagging:")
for token in doc:
    print(f"{token.text:20} ➤ {token.pos_:10} ➤ {token.tag_}")
```

```
 POS Tagging:
Operation            ➤ PROPN      ➤ NNP
Sindoor              ➤ PROPN      ➤ NNP
was                  ➤ AUX        ➤ VBD
carried              ➤ VERB       ➤ VBN
out                  ➤ ADP        ➤ RP
on                   ➤ ADP        ➤ IN
May                  ➤ PROPN      ➤ NNP
7                    ➤ NUM        ➤ CD
,                    ➤ PUNCT      ➤ ,
2025                 ➤ NUM        ➤ CD
,                    ➤ PUNCT      ➤ ,
between              ➤ ADP        ➤ IN
1:04                 ➤ NUM        ➤ CD
AM                   ➤ NOUN       ➤ NN
and                  ➤ CCONJ      ➤ CC
1:24                 ➤ NUM        ➤ CD
AM                   ➤ NOUN       ➤ NN
.                    ➤ PUNCT      ➤ .
It                   ➤ PRON       ➤ PRP
involved             ➤ VERB       ➤ VBD
precision            ➤ NOUN       ➤ NN
strikes              ➤ NOUN       ➤ NNS
against              ➤ ADP        ➤ IN
```

10. Noun Phrase Chunking:

```
[25] print("\n Syntax (Dependency Parsing):")
     for token in doc:
         print(f"{token.text:20} ➤ {token.dep_:15} ➤ Head: {token.head.text}")
```

```
 Syntax (Dependency Parsing):
Operation            ➤ compound        ➤ Head: Sindoor
Sindoor              ➤ nsubjpass       ➤ Head: carried
was                  ➤ auxpass         ➤ Head: carried
carried              ➤ ROOT            ➤ Head: carried
out                  ➤ prt             ➤ Head: carried
on                   ➤ prep            ➤ Head: carried
May                  ➤ pobj            ➤ Head: on
7                    ➤ nummod          ➤ Head: May
,                    ➤ punct           ➤ Head: May
2025                 ➤ appos           ➤ Head: May
,                    ➤ punct           ➤ Head: May
between              ➤ prep            ➤ Head: carried
1:04                 ➤ nummod          ➤ Head: AM
AM                   ➤ pobj            ➤ Head: between
and                  ➤ cc              ➤ Head: AM
1:24                 ➤ nummod          ➤ Head: AM
AM                   ➤ conj            ➤ Head: AM
.                    ➤ punct           ➤ Head: carried
It                   ➤ nsubj           ➤ Head: involved
involved             ➤ ROOT            ➤ Head: involved
```

11. Dependency Parsing:

12. Semantics: Named Entity Recognition

```
[27] # 8. Semantics: Named Entity Recognition
     print("\n Named Entities:")
     for ent in doc.ents:
         print(f"{ent.text:40} ➤ {ent.label_}")
```

```
 Named Entities:
May 7, 2025                              ➤ DATE
1:04 AM                                  ➤ TIME
1:24 AM                                  ➤ TIME
nine                                     ➤ CARDINAL
Pakistan                                 ➤ GPE
Pakistan                                 ➤ GPE
Jammu                                    ➤ GPE
Kashmir                                  ➤ LOC
PoJK                                     ➤ ORG
Lashkar-e-Taiba                          ➤ PERSON
Jaish                                    ➤ PERSON
SCALP Missiles                           ➤ PERSON
Loitering Munitions                      ➤ ORG
```

Practical no 2

Probability estimation in NLP using N-gram models

Introduction to N-gram

An **n-gram** is simply a **sequence of *n* items** from a given text or speech.
These items can be characters, syllables, or most commonly, **words**. N-grams are widely used in **Natural Language Processing (NLP)** to analyze text patterns, predict words, and model language.

Definition

- If *n = 1* → **Unigram** → single word (e.g., "The", "dog")
- If *n = 2* → **Bigram** → two consecutive words (e.g., "The dog", "dog barks")
- If *n = 3* → **Trigram** → three consecutive words (e.g., "The dog barks")
- General case → **n-gram** → sequence of *n* consecutive items.

Example

Sentence: "I love programming"

- **Unigrams**: ["I", "love", "programming"]
- **Bigrams**: [("I", "love"), ("love", "programming")]
- **Trigrams**: [("I", "love", "programming")]

Applications of N-grams

1. **Text Prediction**
    - Predict the next word based on the previous *(n–1)* words.
      Example: “I am going to” → likely next word: “school”.
2. **Spelling and Grammar Correction**
    - Detect unlikely word combinations.
3. **Search Engines**
    - Match phrases and improve query suggestions.
4. **Machine Translation**

- Preserve context by looking at word sequences.

5. **Sentiment Analysis**
   - Capture patterns that single words miss.

Advantages

- Easy to implement and understand.
- Captures some context in text.

Limitations

- Larger *n* means more memory and data requirements.
- Doesn't capture long-distance relationships between words.
- Suffers from **data sparsity** when *n* is large.

1. General n-gram Probability Formula

The probability of a sequence of words $w_1, w_2, \ldots, w_m$ using an **n-gram model** is:

$$P(w_1, w_2, \ldots, w_m) \approx \prod_{k=1}^{m} P(w_k \mid w_{k-(n-1)}, \ldots, w_{k-1})$$

Where:

- n = number of words in the n-gram
- $P(w_k \mid \ldots)$ = conditional probability of the current word given the previous $n-1$ words

2. Specific Cases

Unigram (n = 1)

Assumes each word is independent:

$$P(w_1, w_2, \ldots, w_m) \approx \prod_{k=1}^{m} P(w_k)$$

Probability is simply the frequency of the word divided by total number of words:

$$P(w) = \frac{\text{Count}(w)}{\text{Total number of words}}$$

Bigram (n = 2)

Considers the probability of a word given the previous word:

$$P(w_1, w_2, \ldots, w_m) \approx \prod_{k=1}^{m} P(w_k \mid w_{k-1})$$

Calculated as:

$$P(w_k \mid w_{k-1}) = \frac{\text{Count}(w_{k-1}, w_k)}{\text{Count}(w_{k-1})}$$

Trigram (n = 3)

Considers the probability of a word given the previous two words:

$$P(w_1, w_2, \ldots, w_m) \approx \prod_{k=1}^{m} P(w_k \mid w_{k-2}, w_{k-1})$$

Calculated as:

$$P(w_k \mid w_{k-2}, w_{k-1}) = \frac{\text{Count}(w_{k-2}, w_{k-1}, w_k)}{\text{Count}(w_{k-2}, w_{k-1})}$$

Code:

1. **Importing Libraries:**

✓ 4s

```
import nltk
from nltk import ngrams
from nltk.tokenize import word_tokenize
from collections import Counter
import string
```

2. Download tokenizer data:

```
[9]
# Download tokenizer data
nltk.download('punkt_tab')
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

3. Example Sentence:

```
[36]
# Example sentence
sentence = """What is NLP?
Natural Language Processing (NLP) is a field of Artificial Intelligence (AI) and Li
"""
```

4. Tokenize and remove punctuation:

```
# Tokenize and remove punctuation
tokens = [
    word
    for word in word_tokenize(sentence)
    if word not in string.punctuation
    ]
```

```
[38] print(f"Tokens: {tokens}")
```

```
Tokens: ['What', 'is', 'NLP', 'Natural', 'Language', 'Processing', 'NLP', 'is', 'a'
```

5. Generate n-grams:

```
# --- Generate n-grams ---
unigrams = list(ngrams(tokens, 1))
bigrams = list(ngrams(tokens, 2))
trigrams = list(ngrams(tokens, 3))
```

```
[40] print(f"Unigrams: {unigrams}")
     print(f"Bigrams: {bigrams}")
     print(f"Trigrams: {trigrams}")
```

```
Unigrams: [('What',), ('is',), ('NLP',), ('Natural',), ('Language',), ('Processing
Bigrams: [('What', 'is'), ('is', 'NLP'), ('NLP', 'Natural'), ('Natural', 'Language
Trigrams: [('What', 'is', 'NLP'), ('is', 'NLP', 'Natural'), ('NLP', 'Natural', 'La
```

6. Count Frequencies:

```
# --- Count frequencies ---
unigram_counts = Counter(unigrams)
bigram_counts = Counter(bigrams)
trigram_counts = Counter(trigrams)

total_unigrams = sum(unigram_counts.values())
```

```
[42] print(f"No. of unigram - {unigram_counts}")
     print(f"No. of bigram - {bigram_counts}")
     print(f"No. of trigram - {trigram_counts}")

     print(f"Total no. of unigrams - {total_unigrams}")
```

```
No. of unigram - Counter({('is',): 2, ('NLP',): 2, ('and',): 2, ('What',): 1
No. of bigram - Counter({('What', 'is'): 1, ('is', 'NLP'): 1, ('NLP', 'Natur
No. of trigram - Counter({('What', 'is', 'NLP'): 1, ('is', 'NLP', 'Natural')
Total no. of unigrams - 28
```

7. Calculate probabilities:

```
[43]
# --- Calculate probabilities ---
# Unigram probability
unigram_prob = {gram: count / total_unigrams for gram, count in unigram_counts.items()}

# Bigram probability
bigram_prob = {gram: count / unigram_counts[(gram[0],)] for gram, count in bigram_counts.items()}

# Trigram probability
trigram_prob = {gram: count / bigram_counts[(gram[0], gram[1])] for gram, count in trigram_counts.items()}
```

8. Output:

```
# --- Print results ---
print("=== Unigrams & Probabilities ===")
for gram, prob in unigram_prob.items():
    print(f"{gram}: {prob:.5f}")

print("\n=== Bigrams & Probabilities ===")
for gram, prob in bigram_prob.items():
    print(f"{gram}: {prob:.3f}")


print("\n=== Trigrams & Probabilities ===")
for gram, prob in trigram_prob.items():
    print(f"{gram}: {prob:.5f}")
```

```
=== Unigrams & Probabilities ===
('What',): 0.03571
('is',): 0.07143
('NLP',): 0.07143
('Natural',): 0.03571
('Language',): 0.03571
('Processing',): 0.03571
('a',): 0.03571
('field',): 0.03571
('of',): 0.03571
('Artificial',): 0.03571
('Intelligence',): 0.03571
('AI',): 0.03571
```